Azure Data Scientist Training — Day 4

Automated ML, Pipelines & MLOps | DP-100 Certification Prep

DAY 4 OF 5

4-HOUR SESSION

Session Overview

Day 4 shifts from manual model building to **automation at scale**. In 4 hours, you'll cover the theory and hands-on skills needed to automate the full ML lifecycle on Azure.

🕐 Hour 1

Theory: AutoML, Pipelines & MLOps concepts

🔬 Hours 2–4

Hands-on Labs 10, 11 & 12

01

Automated Machine Learning (AutoML)

02

ML Pipelines for Workflow Automation

03

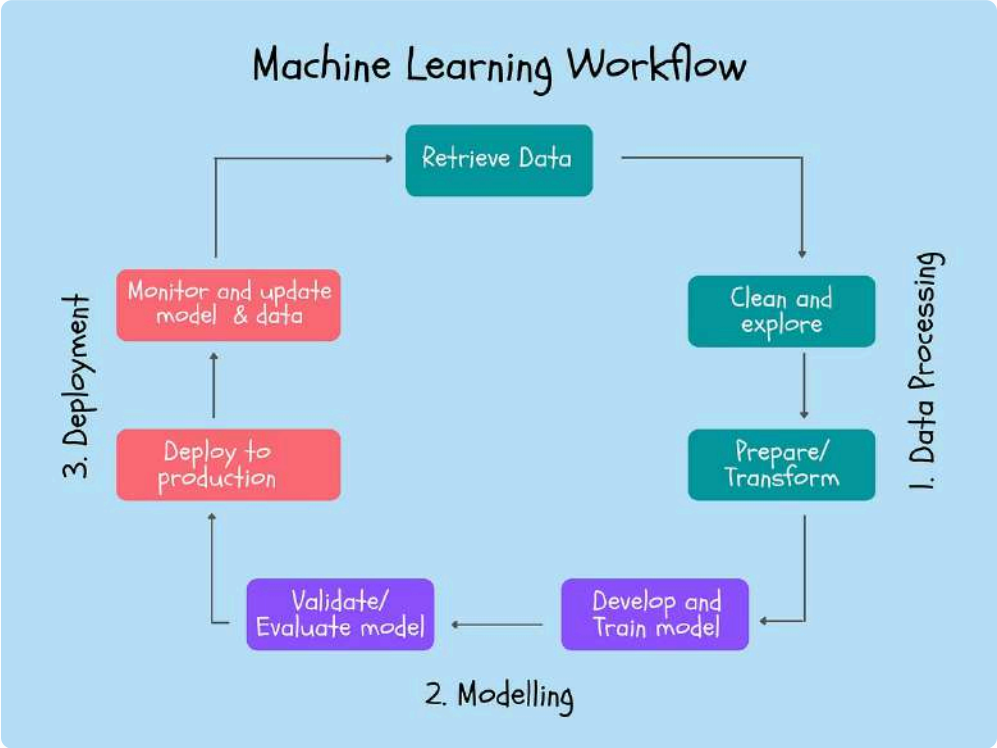
Pipeline Scheduling & Continuous Retraining

04

MLOps: Managing Models in Production

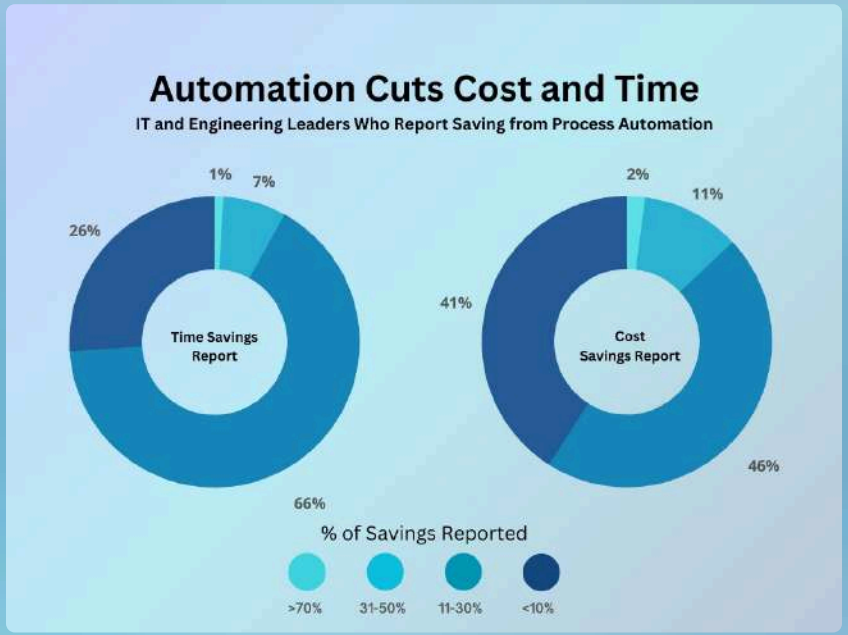
AUTOML

What Is Automated Machine Learning?



AutoML is Azure ML's capability to **automatically explore, train, and tune** ML models — without requiring manual algorithm selection or hyperparameter tuning.

→ You provide the data → Azure ML runs the experiments → You get the best model



Why Use AutoML?

Speed

Trains dozens of models in parallel in minutes

Reduced Expertise Barrier

No deep ML knowledge required to start

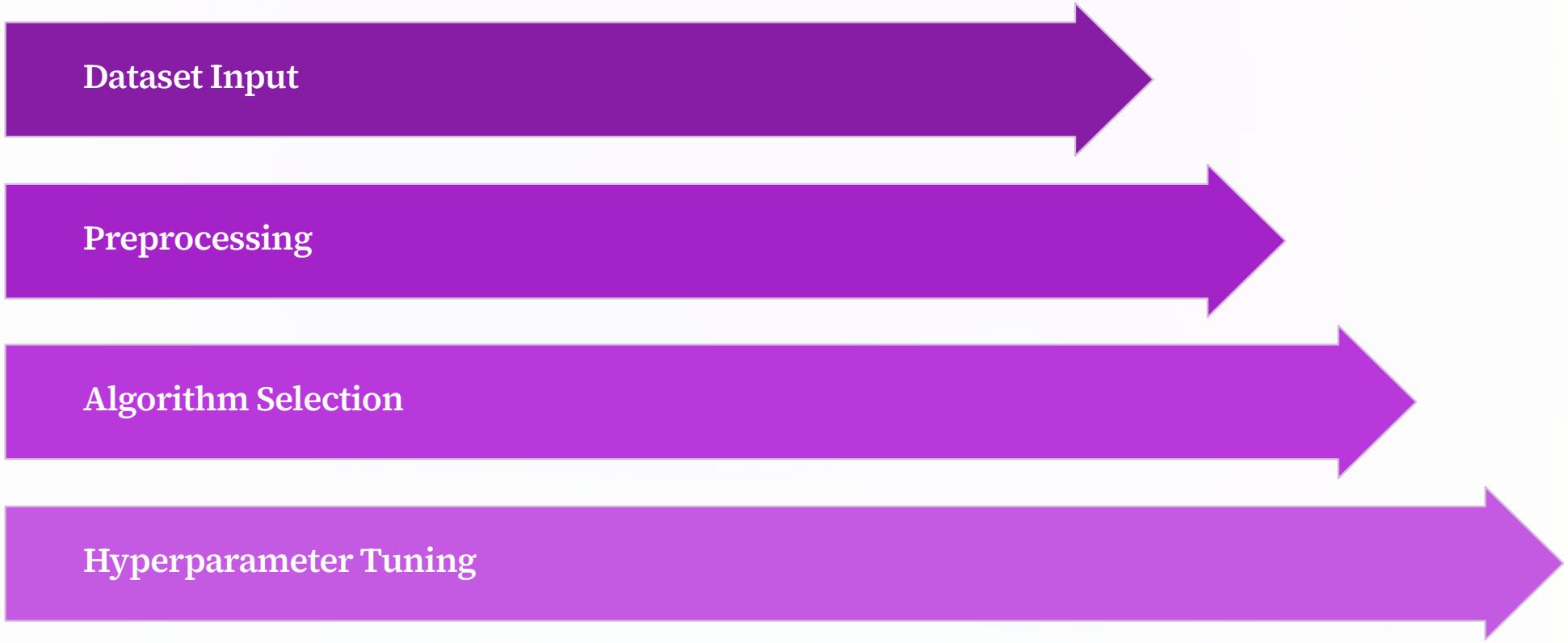
Broad Exploration

Automatically tries many algorithms and featurizations

Best Model Selection

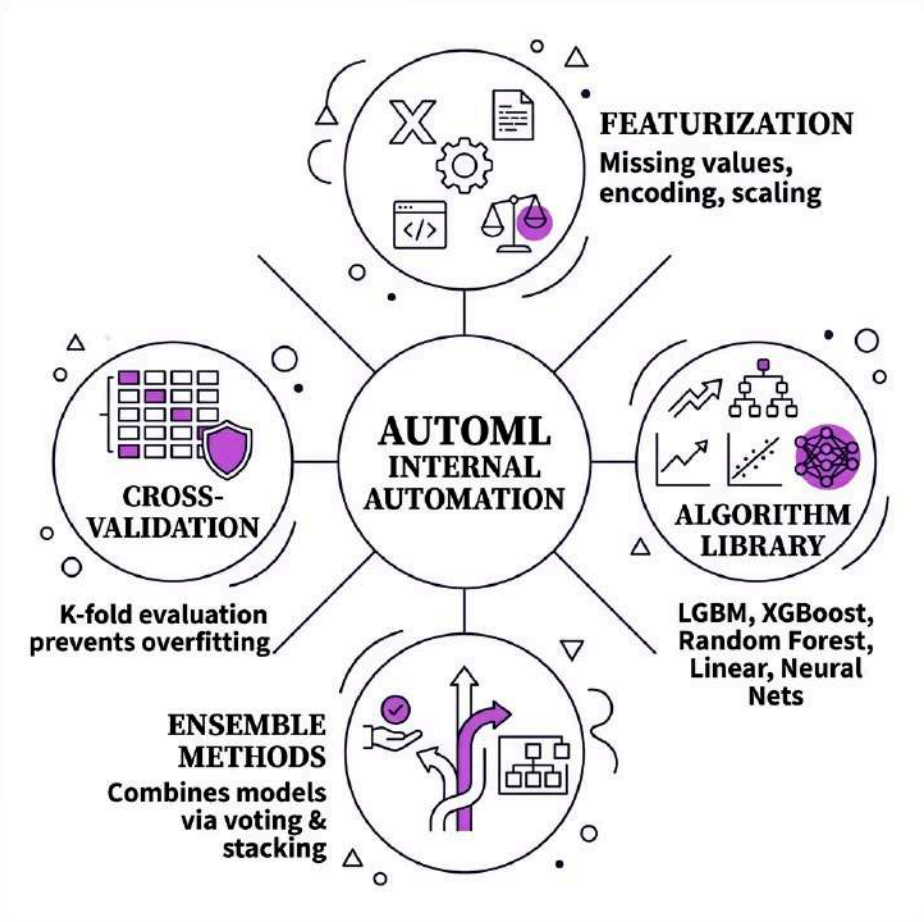
Ranks and surfaces the top-performing model automatically

The AutoML Workflow



Azure ML manages every step automatically — from raw data ingestion to registering the best-performing model in your workspace registry.

AutoML: Under the Hood

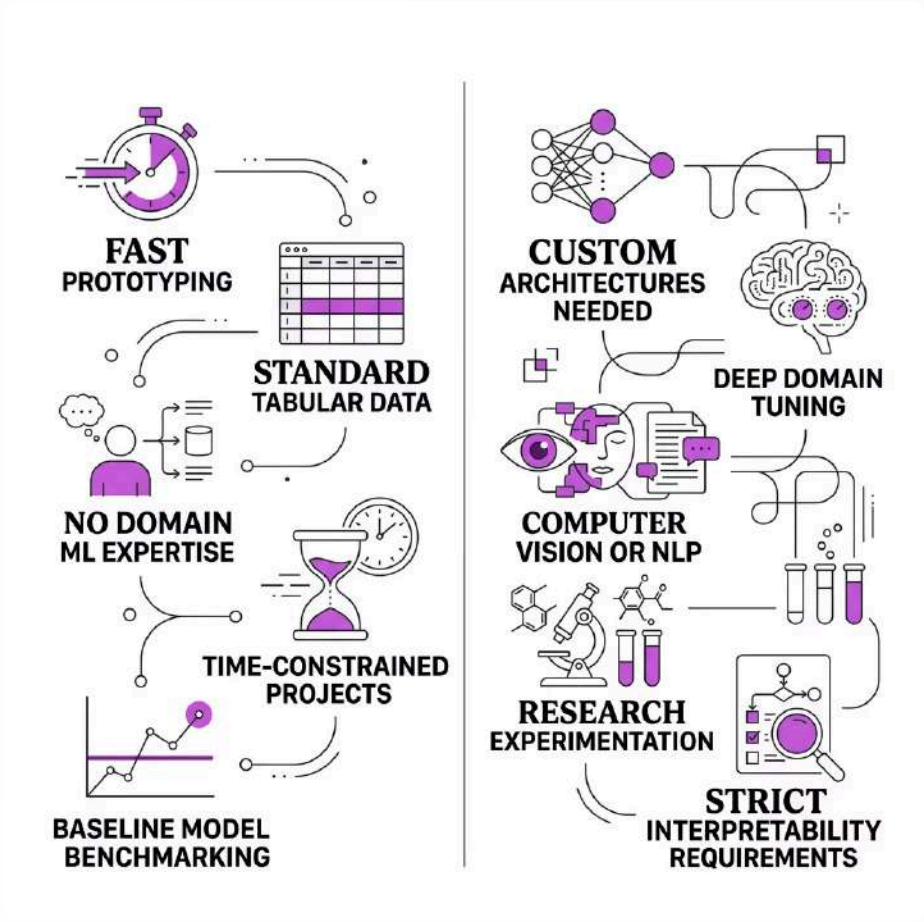


What Happens Automatically

- **Featurization** — data cleaning, encoding, and normalization handled for you
- **Algorithm sweep** — dozens of algorithms evaluated in parallel
- **Ensemble stacking** — top models combined for maximum accuracy
- **Early stopping** — underperforming runs terminated automatically

AutoML vs. Manual ML: When to Use Which?

Choosing Your Approach



💡 **Best practice:** Use AutoML to quickly establish a performance baseline, then hand-tune only if the baseline falls short of business requirements.

Real-World AutoML Use Cases

Customer Churn

Predict which subscribers will cancel — classify customers by churn risk within days, not weeks.

Fraud Detection

Flag suspicious transactions in real time using AutoML classification on historical patterns.

Sales Forecasting

Forecast next quarter's revenue with time-series AutoML — retrained weekly on fresh data.

Demand Prediction

Optimize inventory by predicting product demand across regions and seasons automatically.

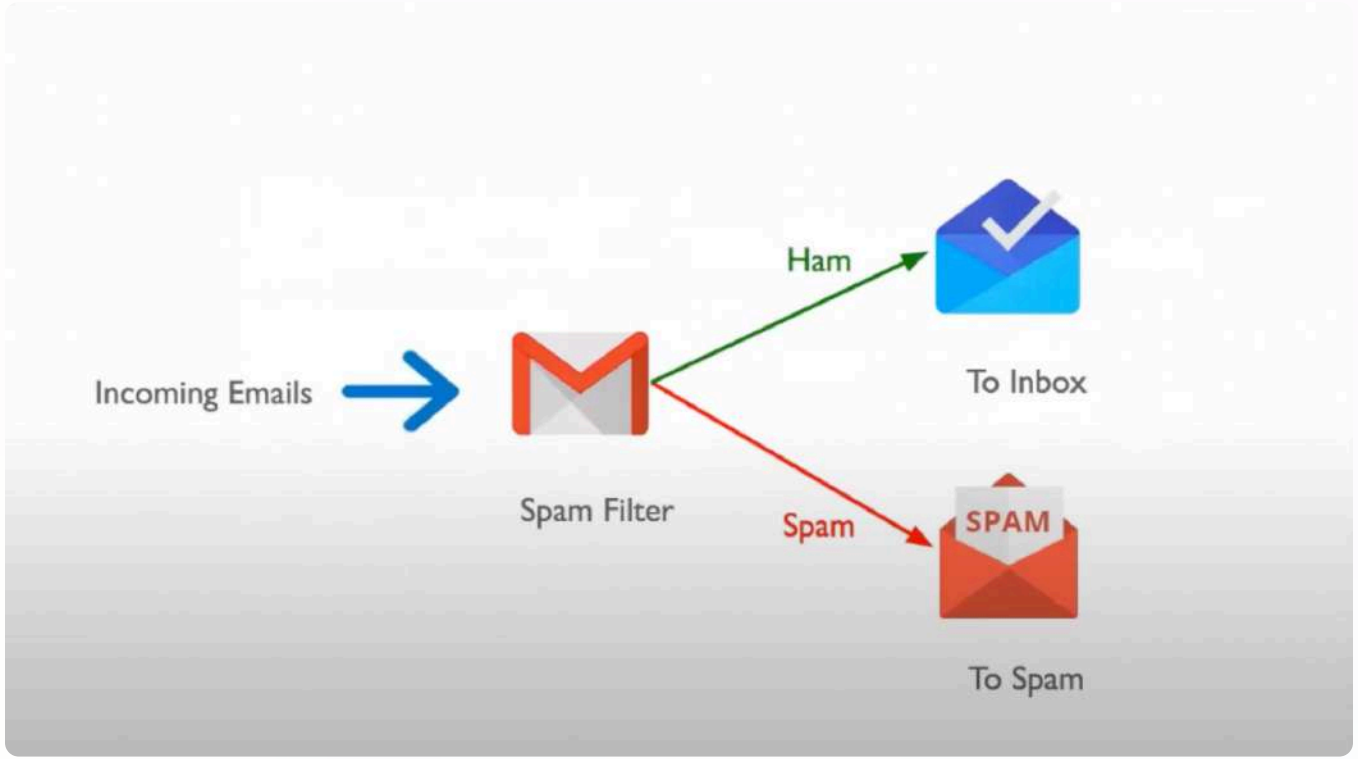
AUTOML TASK TYPES

Three Supported Task Types in Azure AutoML

Classification	Regression	Forecasting
Predict a **discrete category** label	Predict a **continuous numeric** value	Predict **future values** over time

Each task type configures a different set of algorithms, evaluation metrics, and preprocessing strategies within the same AutoML framework.

Task Type: Classification



Predict a Category Label

Use classification when your target column contains **discrete classes** such as Yes/No, categories, or labels.

- Spam vs. Not Spam detection
- Customer churn prediction (True/False)
- Disease diagnosis (Positive/Negative)
- Product category assignment

Primary metrics: AUC, Accuracy, F1 Score, Precision, Recall

Task Type: Regression

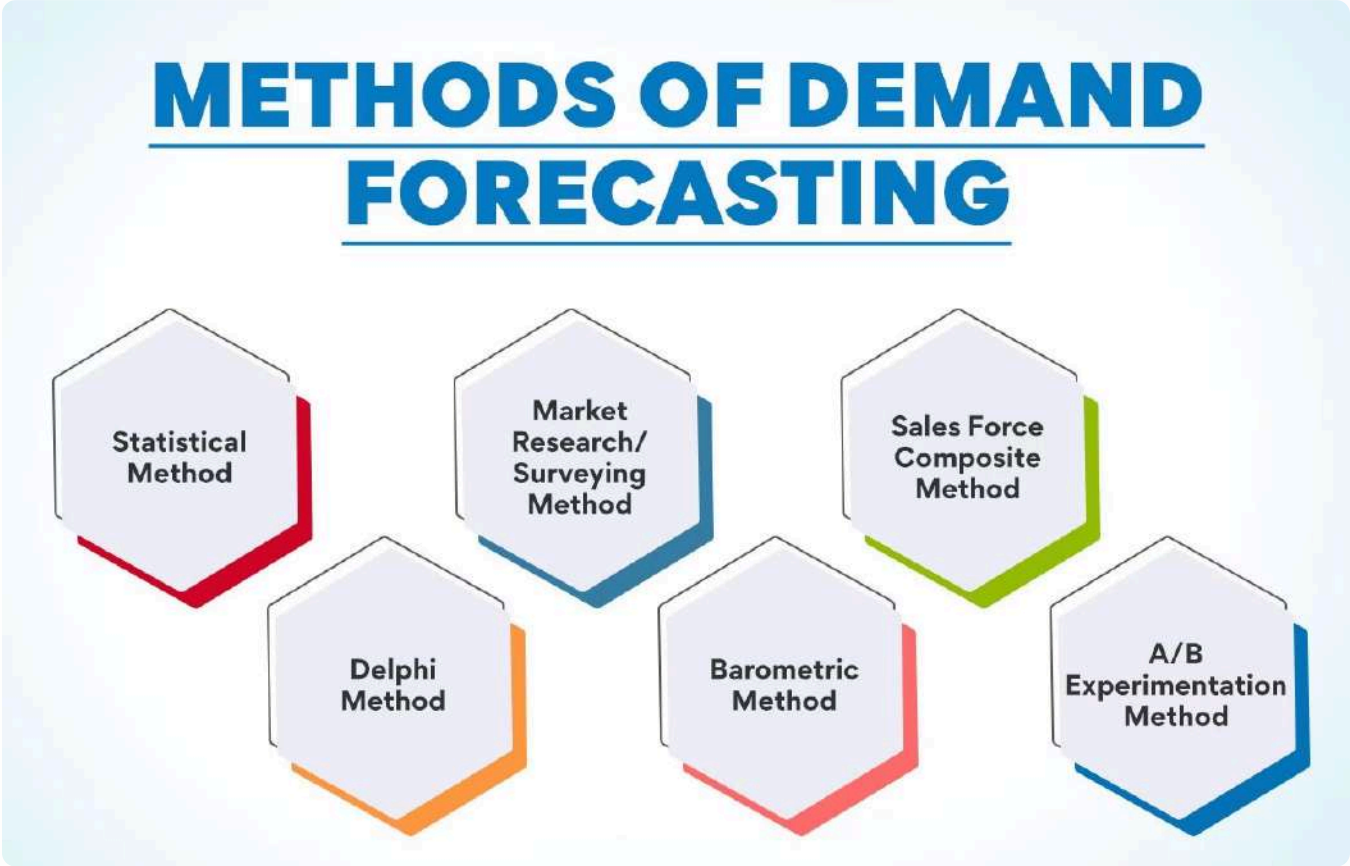
Predict a Continuous Number

Use regression when your target column contains a **numeric value** on a continuous scale.

- House price estimation
- Monthly sales revenue forecast
- Employee attrition rate
- Insurance premium calculation

Primary metrics: RMSE, R^2, MAE, MAPE

Task Type: Forecasting



Predict Future Values Over Time

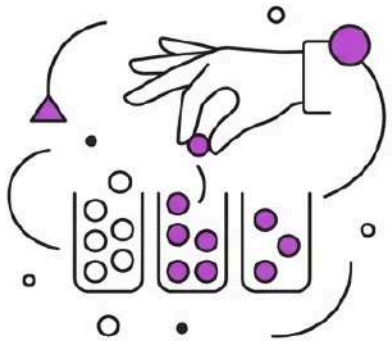
Use forecasting for **time-indexed data** where temporal patterns drive predictions.

- Product demand forecasting
- Monthly energy consumption prediction
- Retail sales time-series prediction
- Website traffic forecasting

Requires: A timestamp column and a defined forecast horizon

AutoML Task Types at a Glance

CLASSIFICATION



PREDICTS DISCRETE LABELS

Algorithms: LightGBM / XGBoost / Logistic Regression

Metrics: AUC / Accuracy / F1

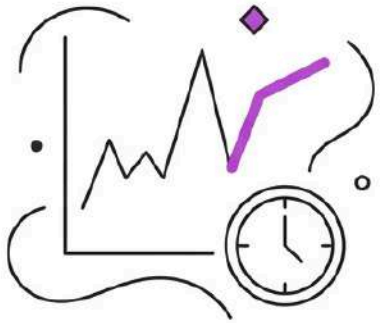
REGRESSION

PREDICTS CONTINUOUS NUMBERS

Algorithms: ElasticNet / Random Forest / Gradient Boosting

Metrics: RMSE / R2 / MAE

FORECASTING



PREDICTS TIME-SERIES VALUES

Algorithms: AutoARIMA / Prophet / TCNForecaster

Metrics: MAPE / SMAPE / Normalized RMSE

Azure AutoML selects the algorithm pool and evaluation metrics automatically based on the task type you specify during experiment configuration.

CONFIGURATION

Configuring an AutoML Experiment

1	**Select Dataset** Register a tabular dataset in your Azure ML workspace
2	**Define Target Column** Specify the column AutoML should learn to predict
3	**Choose Compute** Attach a CPU compute cluster to run parallel trials
4	**Set Primary Metric** Pick the optimization metric (e.g., AUC, RMSE)
5	**Set Time Limit** Define max experiment duration to control costs

AutoML Configuration: Key Parameters

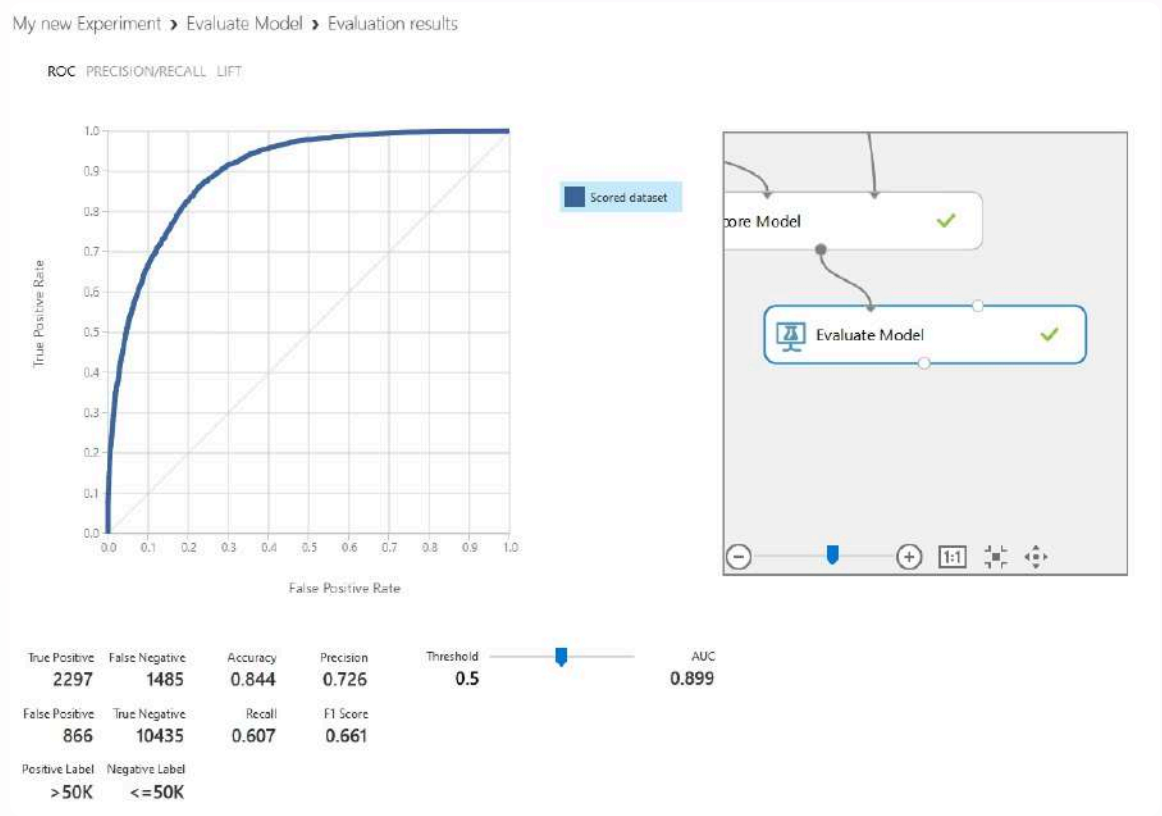
SDK Configuration Example

```
from azureml.train.automl import AutoMLConfig

automl_config = AutoMLConfig(
    task='classification',
    primary_metric='AUC_weighted',
    training_data=train_dataset,
    label_column_name='churn',
    compute_target=cpu_cluster,
    experiment_timeout_minutes=30,
    max_concurrent_iterations=4,
    featurization='auto',
    enable_early_stopping=True
)
```

Key Parameters Explained

- `task` — classification, regression, or forecasting
- `primary_metric` — what AutoML optimizes
- `experiment_timeout_minutes` — hard cost cap
- `max_concurrent_iterations` — parallel trials (match cluster nodes)
- `featurization='auto'` — enables automatic data preprocessing
- `enable_early_stopping` — terminates poor-performing runs

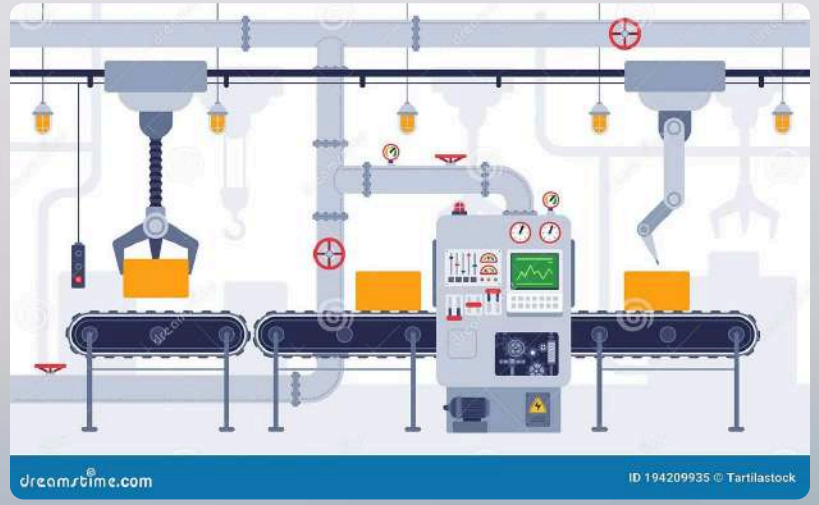
Evaluating AutoML Results



What to Review After a Run

- **Model leaderboard** — ranked list of all tested algorithms
- **Best model metrics** — primary and secondary metric scores
- **Confusion matrix** — classification error breakdown
- **Feature importance** — which columns drove predictions
- **Explainability dashboard** — interpretable AI via SHAP values

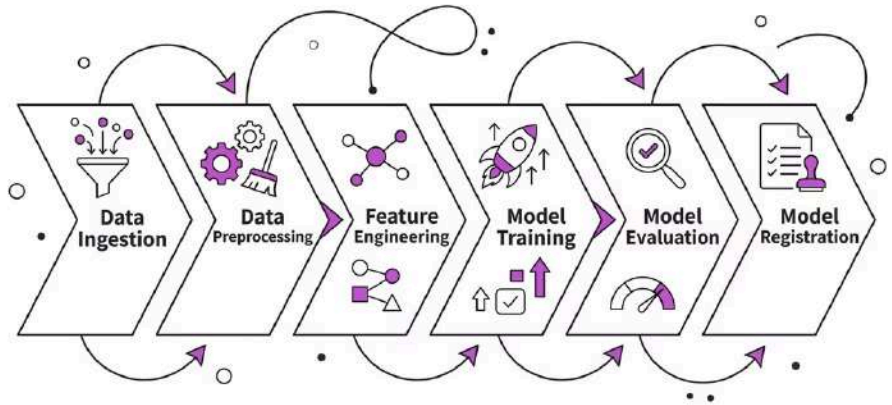
The best model is automatically registered in the Azure ML model registry for deployment.



Chapter 2: ML Pipelines

Automating the full machine learning lifecycle end-to-end

What Is an ML Pipeline?



An **ML Pipeline** in Azure Machine Learning is a reusable, automated workflow that chains ML tasks together — each running as an independent, trackable step.

- Each step can run on different compute targets
- Steps execute in a defined dependency order
- Output from one step feeds the next automatically
- Pipelines are versioned and repeatable

Why Use ML Pipelines?

Reproducibility

Run the exact same workflow weeks later and get consistent results — every step is versioned and logged.

Modularity

Update a single step (e.g., swap the model) without rebuilding the entire pipeline from scratch.

Parallel Execution

Independent steps run in parallel on separate compute clusters, reducing total runtime.

Schedulability

Trigger pipelines on a cron schedule or on data arrival to automate continuous model retraining.

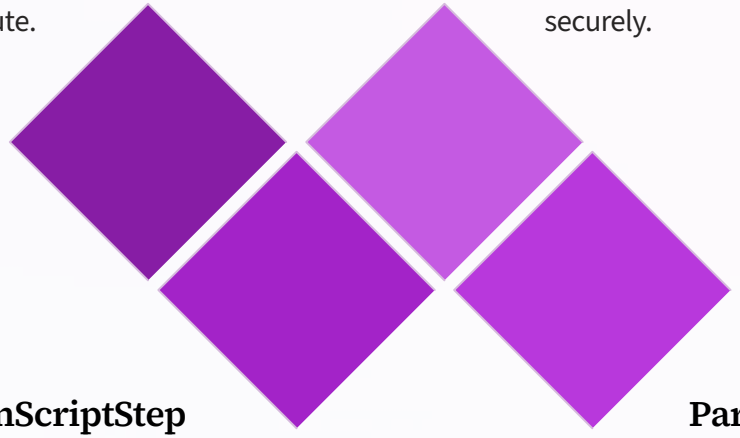
Pipeline Steps: Core Concepts

EstimatorStep

Executes a training estimator on compute.

DataTransferStep

Moves data between datastores securely.



PythonScriptStep

Runs a .py script with arguments.

ParallelRunStep

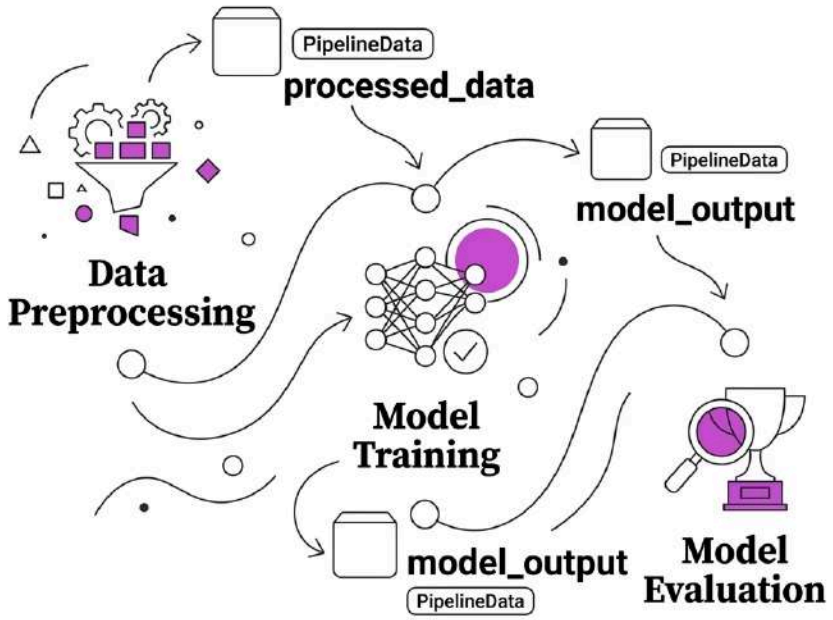
Scores large datasets in parallel batches.

Each step in an Azure ML Pipeline wraps a script or notebook, specifies its inputs/outputs, and declares its compute target — making steps independently testable and reusable across pipelines.

Building Pipeline Steps — SDK Example

```
from azureml.pipeline.steps import PythonScriptStep
from azureml.pipeline.core import Pipeline, PipelineData

# Define intermediate data
processed_data = PipelineData("processed_data", datastore=ws.get_default_datastore())

# Step 1: Data Preprocessing
prep_step = PythonScriptStep(
 name="data-preprocessing",
 script_name="prep.py",
 arguments=["--output", processed_data],
 outputs=[processed_data],
 compute_target=cpu_cluster,
 source_directory="./scripts"
)

# Step 2: Model Training
train_step = PythonScriptStep(
 name="model-training",
 script_name="train.py",
 arguments=["--input", processed_data],
 inputs=[processed_data],
 compute_target=cpu_cluster,
 source_directory="./scripts"
)
```

Data Dependencies Between Steps



How Dependencies Work

Azure ML resolves execution order automatically by tracing **PipelineData** objects passed between steps.

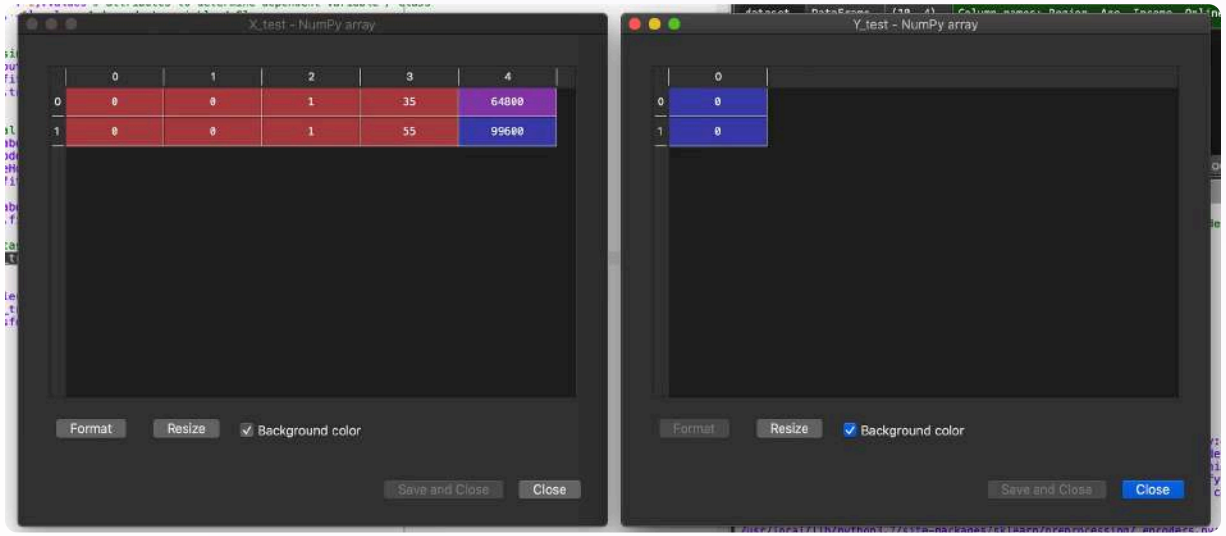
- If Step B uses Step A's output → Step A runs first
- Steps with no shared data can run in parallel
- Cached step results are reused when inputs haven't changed

PipelineData is the key object — it acts as both an output reference and an input reference across steps.

Assembling and Running a Pipeline

```
from azureml.pipeline.core import Pipeline
from azureml.core import Experiment

# Assemble pipeline
pipeline = Pipeline(
 workspace=ws,
 steps=[prep_step, train_step, eval_step]
)

# Validate the pipeline graph
pipeline.validate()

# Submit pipeline as an experiment run
experiment = Experiment(ws, "day4-pipeline")
run = experiment.submit(pipeline)
run.wait_for_completion(show_output=True)

# Publish pipeline as a REST endpoint for reuse
published_pipeline = pipeline.publish(
 name="retraining-pipeline",
 description="Daily retraining pipeline v1"
)
```

Pipeline Step: Data Preprocessing

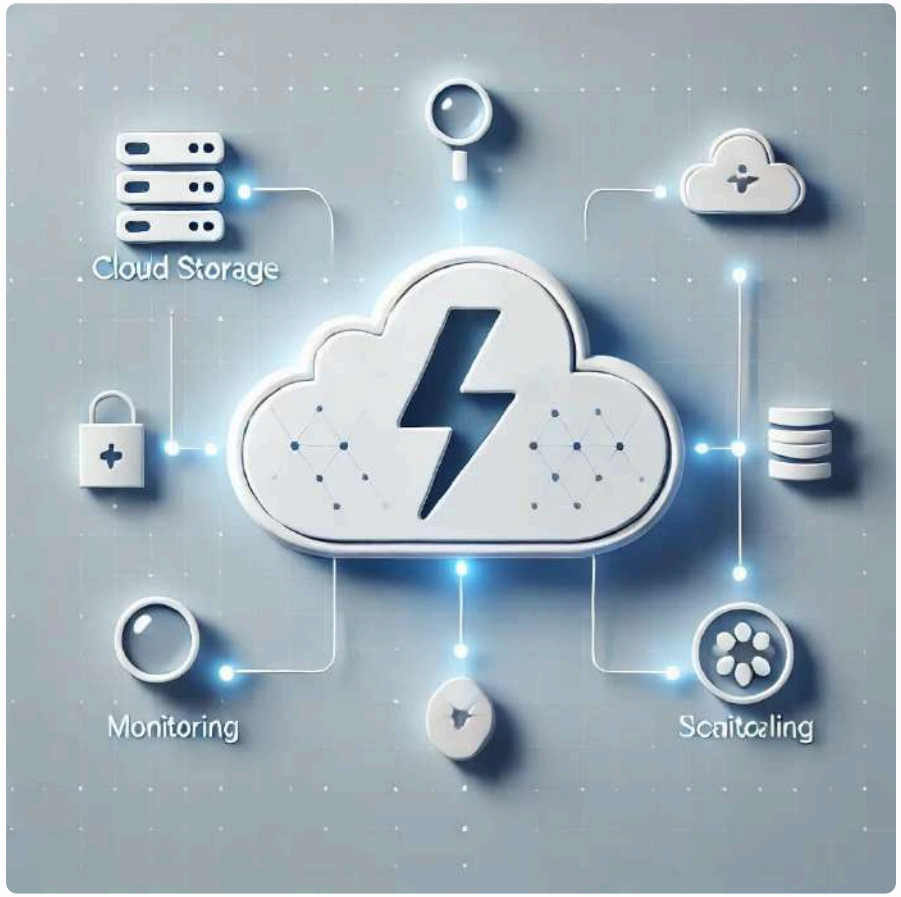


What prep.py Typically Does

- Load raw data from Azure Blob or registered dataset
- Handle missing values and outliers
- Encode categorical variables (one-hot, label)
- Normalize or standardize numeric features
- Split into train/test sets
- Write processed output to `PipelineData`

Separating preprocessing into its own step means it can be **cached and skipped** on subsequent runs when the raw data hasn't changed.

Pipeline Step: Model Training

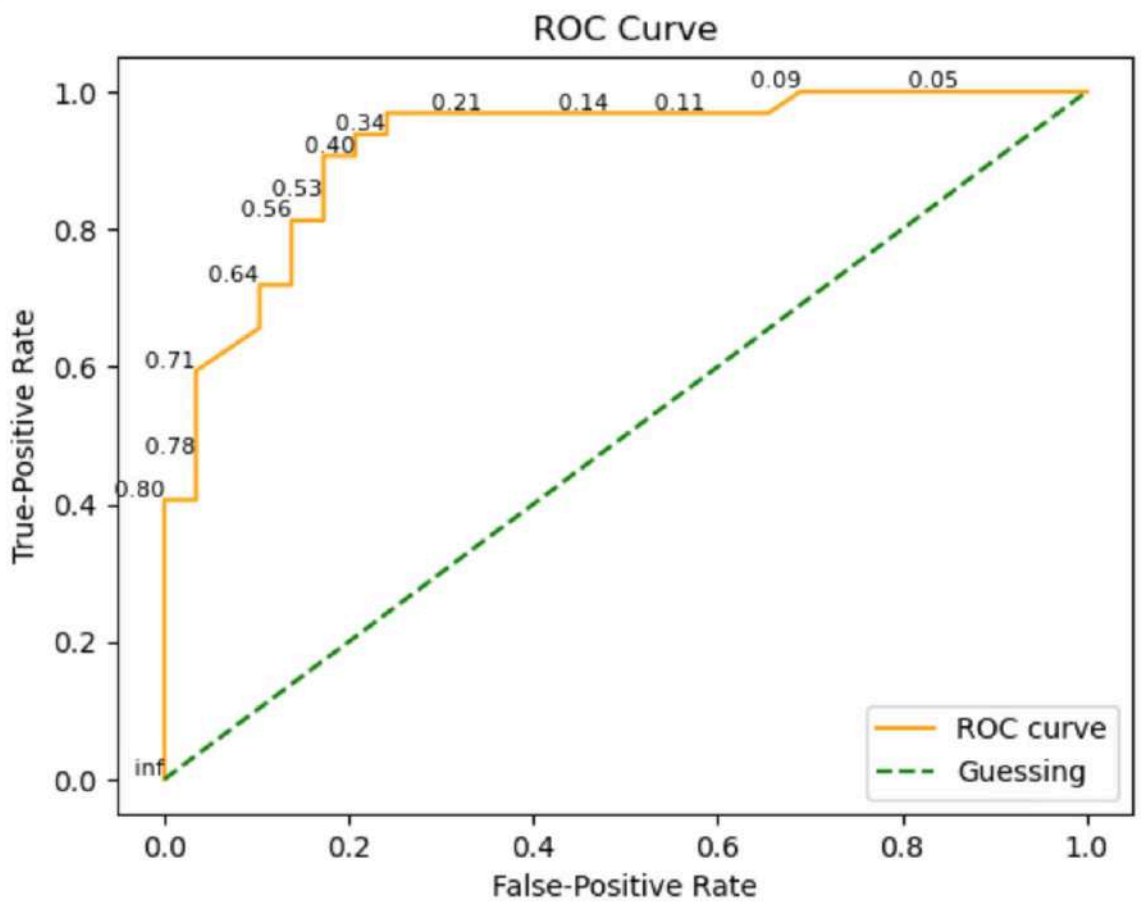


What train.py Typically Does

- Load processed data from previous step's output
- Initialize and fit a scikit-learn or PyTorch model
- Log metrics to Azure ML Run context
- Save the trained model artifact to outputs
- Register the model in Azure ML model registry

Use `run.log()` and `run.log_metric()` inside the script so metrics appear in Azure ML Studio experiment views.

Pipeline Step: Model Evaluation



What eval.py Typically Does

- Load the trained model and test dataset
- Generate predictions on held-out test data
- Compute accuracy, AUC, RMSE, or other metrics
- Compare against previously registered model version
- Gate model registration — only promote if better

The evaluation step acts as a **quality gate**: if the new model underperforms the current production model, it is not registered.

SCHEDULING

Pipeline Scheduling: Automated Retraining

Published pipelines can be triggered on a **time-based schedule** or on **data-change events** — enabling continuous model retraining without manual intervention.

Configuring a Pipeline Schedule

```
from azureml.pipeline.core.schedule import (
    ScheduleRecurrence, Schedule
)

# Daily recurrence at midnight UTC
recurrence = ScheduleRecurrence(
    frequency="Day",
    interval=1,
    hours=[0],
    minutes=[0]
)

# Attach schedule to published pipeline
schedule = Schedule.create(
    workspace=ws,
    name="daily-retraining",
    pipeline_id=published_pipeline.id,
    experiment_name="day4-pipeline",
    recurrence=recurrence,
    description="Retrain model with latest data daily"
)

print(f"Schedule created: {schedule.id}")
```

Schedule Frequencies Available

Minute / Hour

Near-real-time retraining for high-velocity data

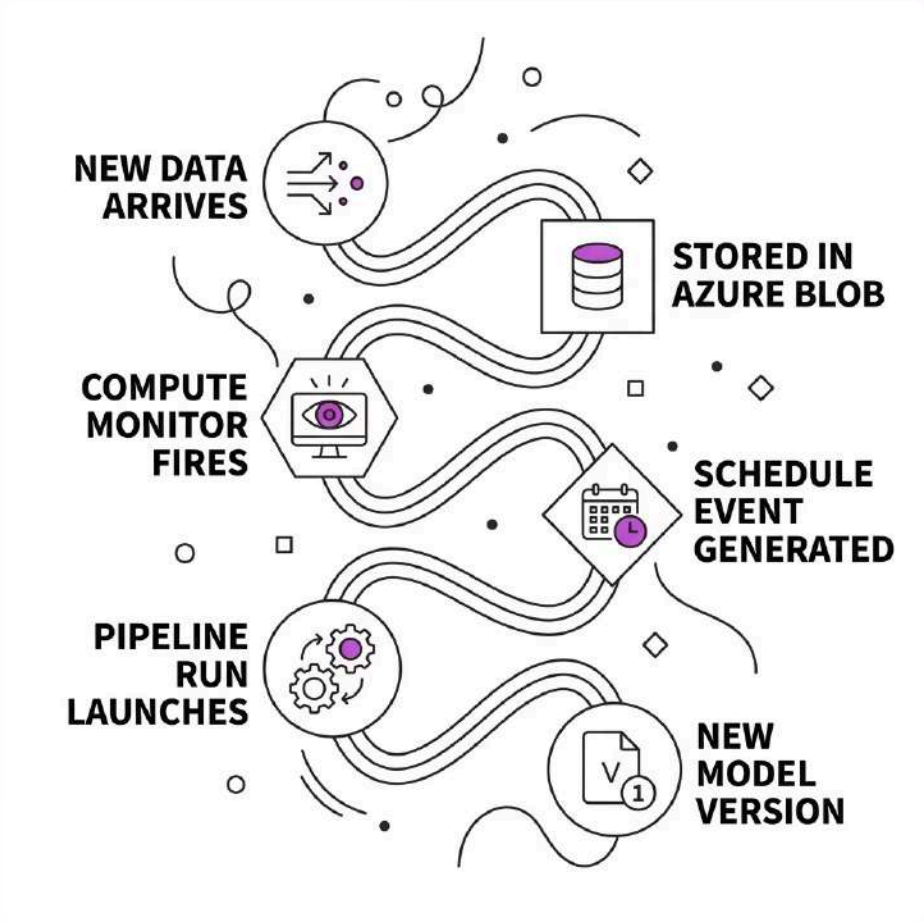
Day

Daily retraining — most common pattern

Week / Month

Batch retraining for slower-moving datasets

Data-Triggered Pipeline Schedules



Trigger on New Data Arrival

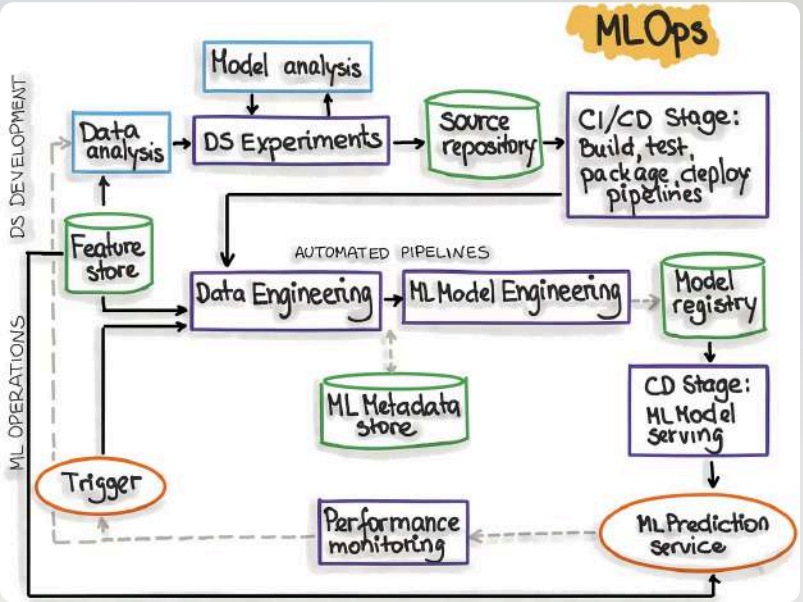
Instead of a fixed clock, you can configure a **datastore-based trigger** that fires whenever new files are deposited in a monitored path.

- Monitor a blob storage path for new CSVs
- Auto-launch retraining on data arrival
- Ideal for event-driven ML architectures

Combine time-based and data-triggered schedules for robust production retraining strategies.

Chapter 3: MLOps

Bringing DevOps discipline to machine learning in production



What Is MLOps?

MLOps applies **DevOps principles** — continuous integration, delivery, and monitoring — to the machine learning lifecycle. It bridges the gap between data science experimentation and reliable production systems.

CI — Continuous Integration

Automated testing of training code, data validation, and model quality on every commit

CT — Continuous Training

Scheduled or triggered retraining pipelines keep models current with new data

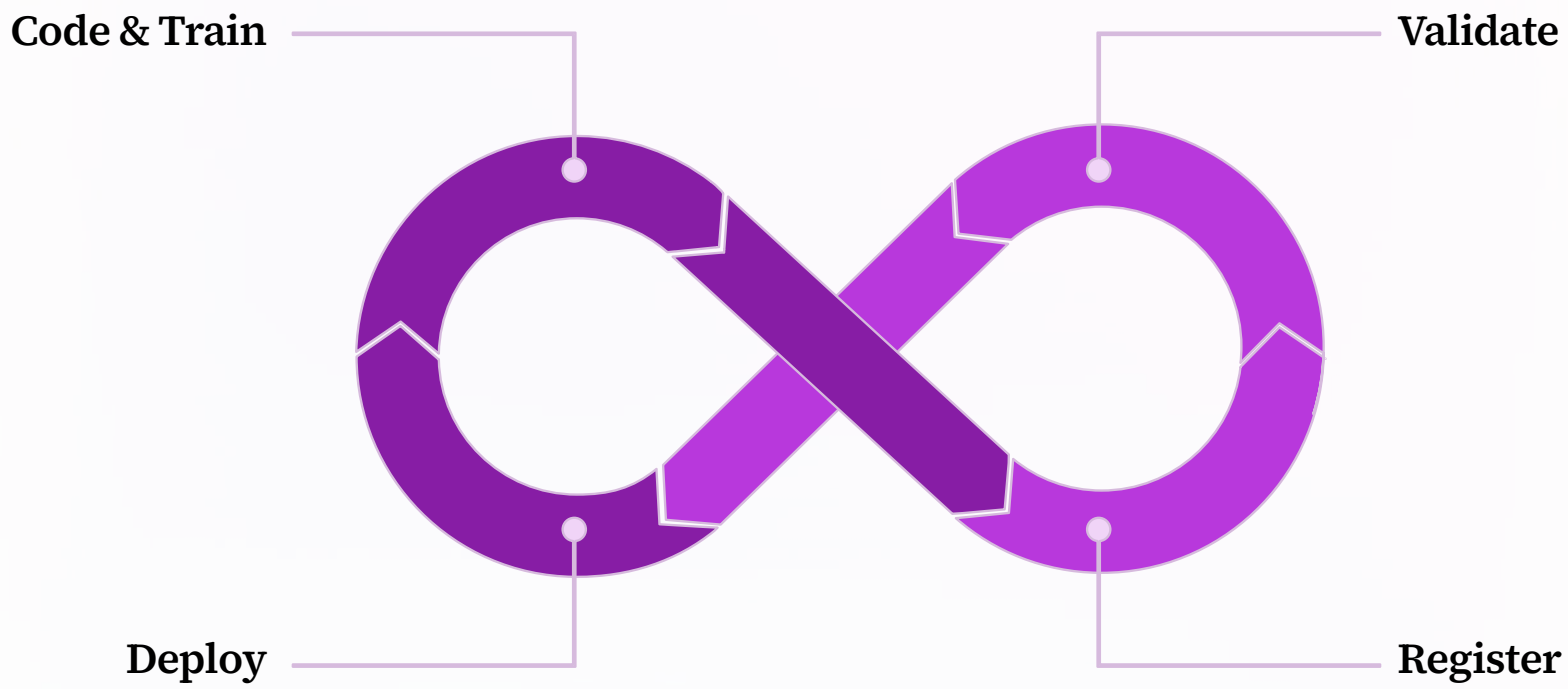
CD — Continuous Deployment

Automated model deployment to staging and production endpoints after passing quality gates

CM — Continuous Monitoring

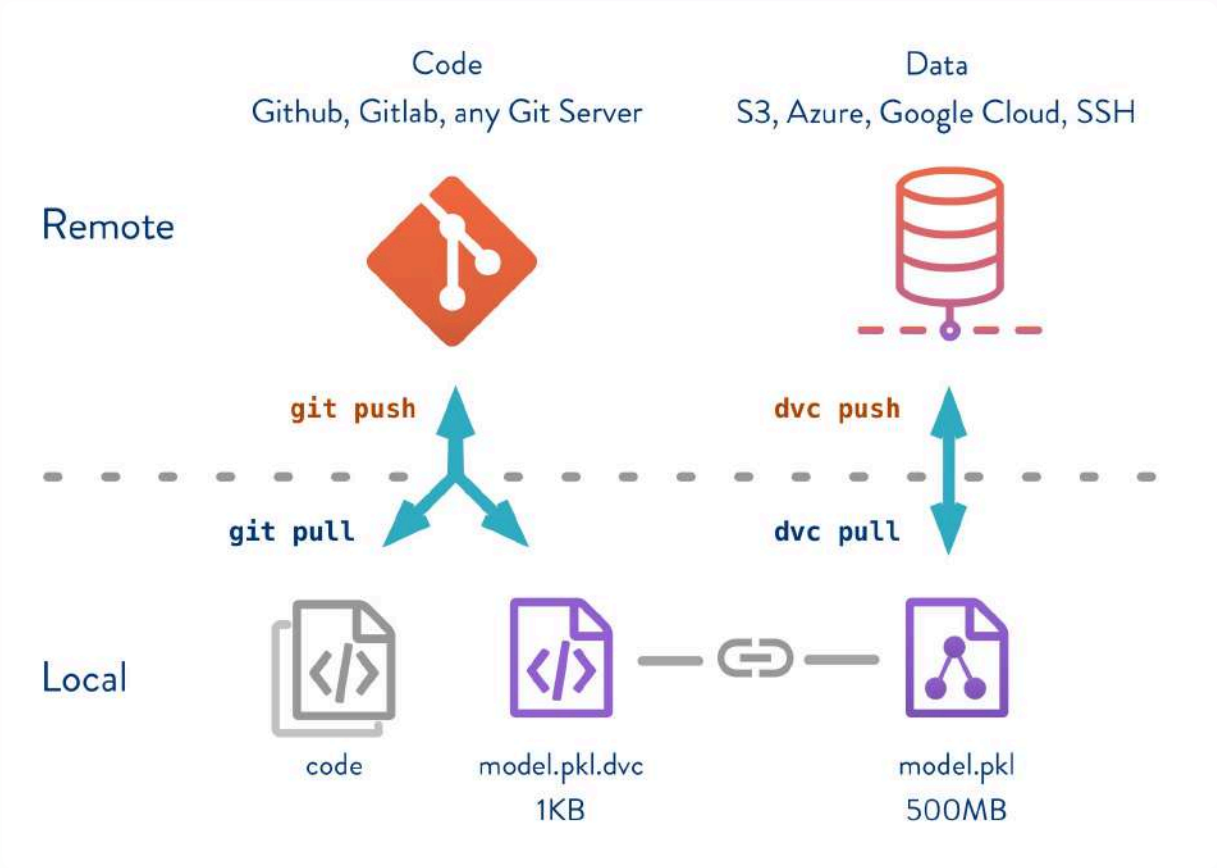
Track data drift, model drift, and endpoint performance in real-time production

The MLOps Lifecycle



MLOps transforms the ML lifecycle from a one-time project into a **continuously improving production system** — every deployment makes the next one faster and safer.

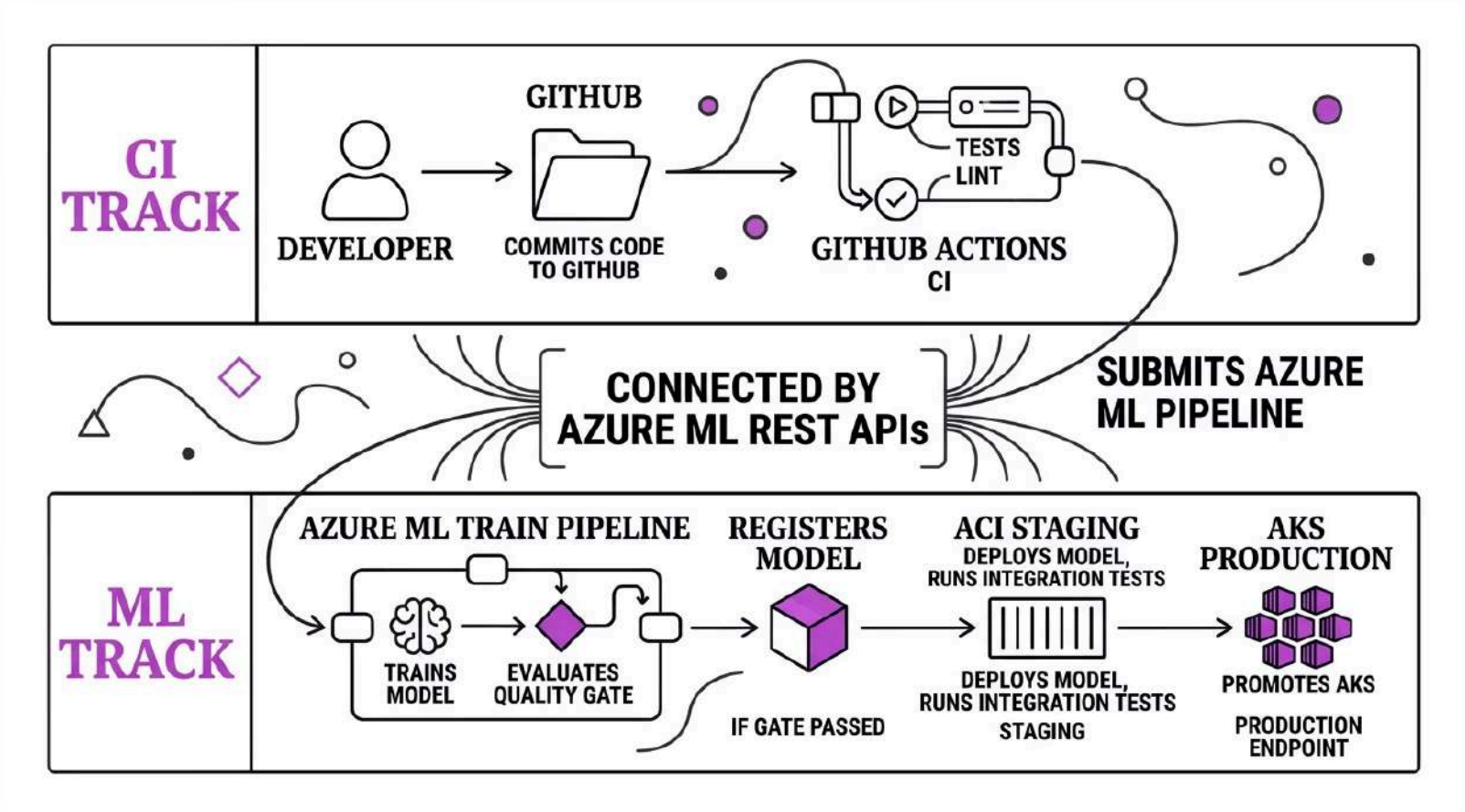
Model Versioning in Azure ML



Model Registry Best Practices

- Every trained model is registered with a **version number** automatically incremented
- Attach tags for `accuracy`, `dataset_version`, `git_commit`
- Compare versions side-by-side in Azure ML Studio
- Promote from **Staging → Production** using model stage transitions
- Roll back to a previous version instantly if degradation is detected

Azure ML + Azure DevOps / GitHub Actions



Integrating Azure ML with GitHub Actions or Azure DevOps Pipelines enables fully automated CI/CT/CD for ML systems — the gold standard for production ML in enterprise environments.

MLOps Maturity Levels

1

Level 0 — Manual

Scripts run locally, models deployed by hand, no versioning

2

Level 1 — Pipeline Automation

ML Pipelines automate training; model registry tracks versions

3

Level 2 — CI/CT Enabled

Code commits trigger automated training and evaluation gates

4

Level 3 — Full CD + Monitoring

Automated deployment, drift detection, and self-healing retraining

Most enterprise teams target **Level 2–3**. Day 4 labs get you from Level 0 to Level 1.

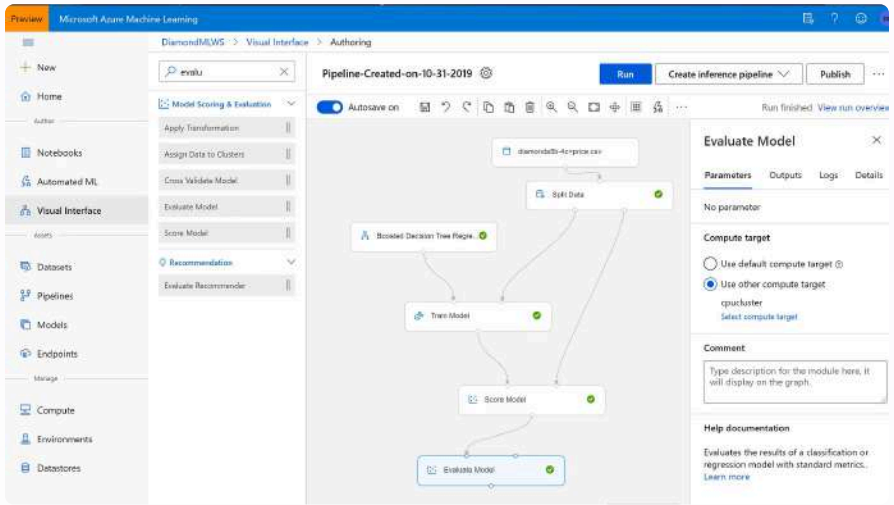
Hands-On Labs

Day 4 — Labs 10, 11 & 12

3 HOURS PRACTICAL

LAB 10

Lab 10 — Build a Model Using AutoML



What You Will Do

01

Upload the customer churn dataset to your workspace

02

Configure AutoML: task=classification, metric=AUC, timeout=20 min

03

Submit the AutoML experiment and monitor runs in Studio

04

Review model leaderboard and inspect the best model

05

Download the best model and review feature importance

Lab 10 — Expected Outcome

10+

Models Tested

AutoML explores over 10 algorithm variants automatically

~20min

Experiment Time

Full AutoML sweep completes within the 20-minute budget

AUC

Primary Metric

Weighted AUC used to rank and select the best classifier

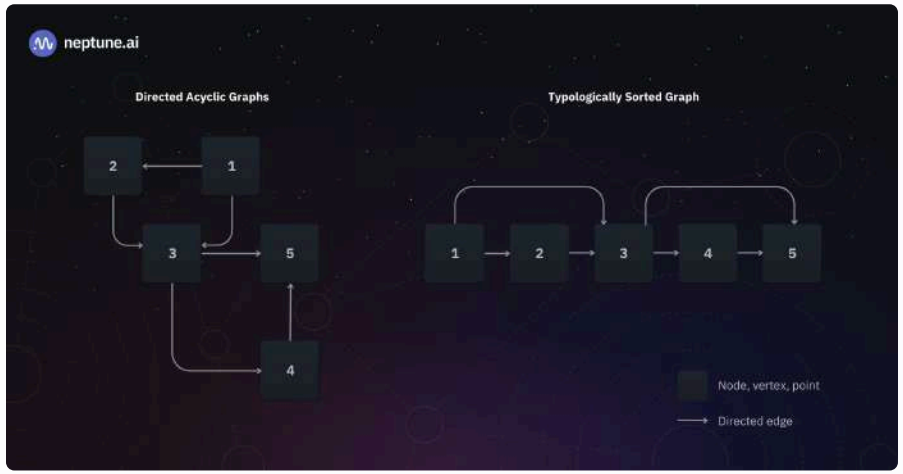
1-click

Model Registration

Best model registered to the workspace in one click

LAB 11

Lab 11 — Create an ML Pipeline



What You Will Do

01

Write prep.py — load dataset, clean, encode, and split

02

Write train.py — load processed data, train classifier, log metrics

03

Write eval.py — evaluate model, compare against baseline

04

Define PythonScriptSteps with PipelineData dependencies

05

Assemble, validate, and run the full pipeline as an experiment

06

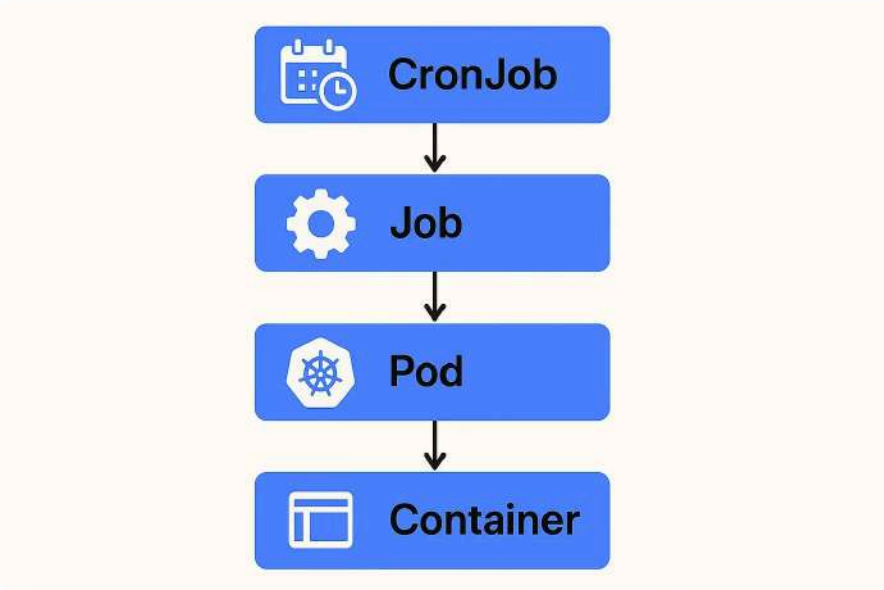
Publish the pipeline as a reusable REST endpoint

Lab 11 — Pipeline Architecture You'll Build

After completing Lab 11, your pipeline will be **published as a REST endpoint** — ready to be triggered programmatically or scheduled in Lab 12.

LAB 12

Lab 12 — Schedule a Pipeline



What You Will Do

01

Use the pipeline published in Lab 11

02

Create a `ScheduleRecurrence` for daily execution

03

Attach the schedule to the published pipeline endpoint

04

Verify the schedule appears in Azure ML Studio

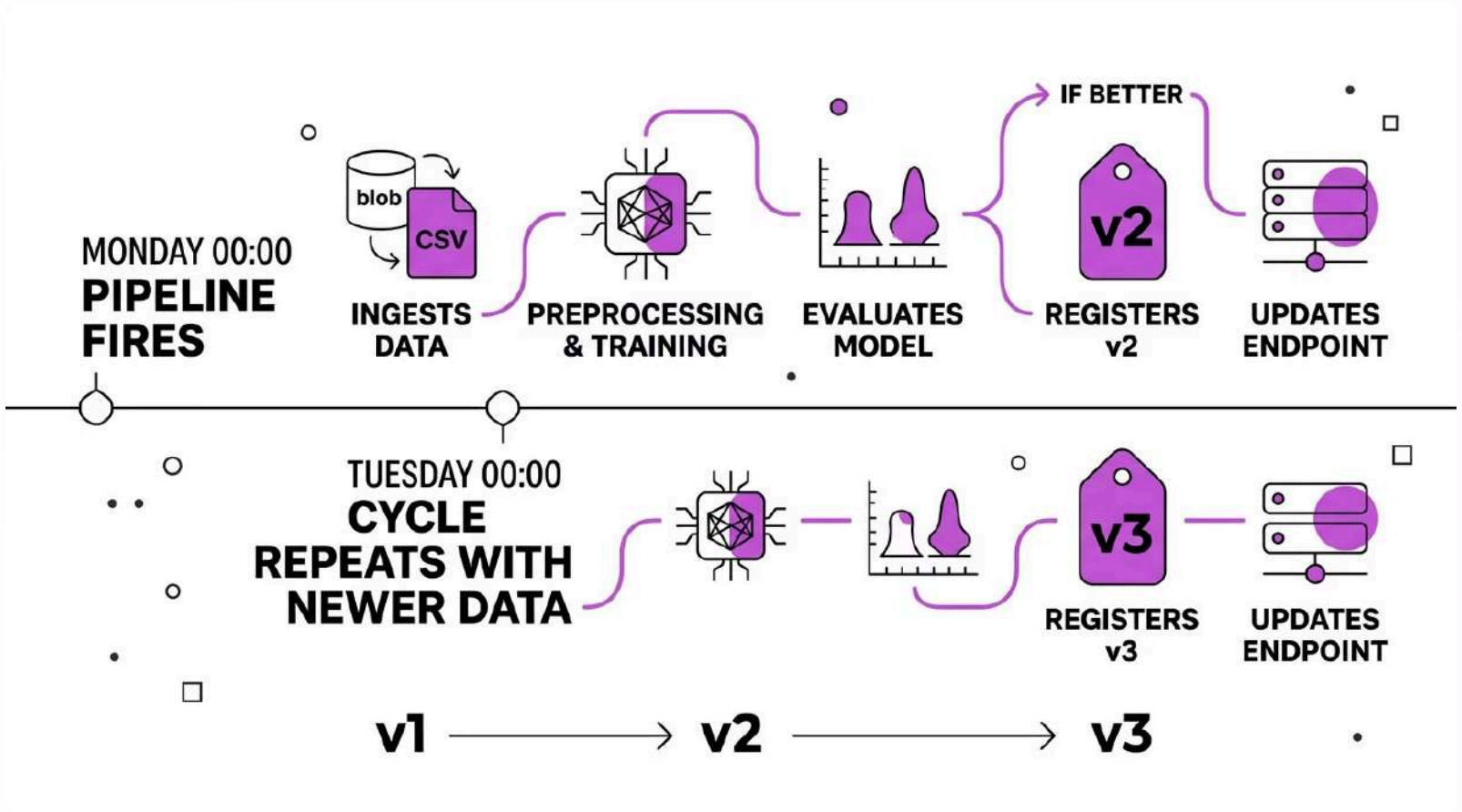
05

Manually trigger and confirm a successful scheduled run

06

Disable the schedule to stop incurring costs after the lab

Lab 12 — What Continuous Retraining Looks Like



This automated daily loop — built in 45 minutes during Lab 12 — represents the **core retraining pattern** used in real enterprise Azure ML deployments.

$ COST OPTIMIZATION

Azure ML Lab Cost Optimization Tips

Keep Your Lab Costs Low

💻 Use CPU, Not GPU

For tabular AutoML and pipeline labs, CPU clusters (e.g., `Standard_DS3_v2`) are sufficient and cost 5–10× less than GPU VMs.

⏱️ Limit AutoML Timeout

Set `experiment_timeout_minutes=20` for lab exercises. Unlimited timeout = unbounded cost. Always cap it.

🔢 Cap Concurrent Iterations

Use `max_concurrent_iterations=2` for labs. More parallelism means more nodes billing simultaneously.

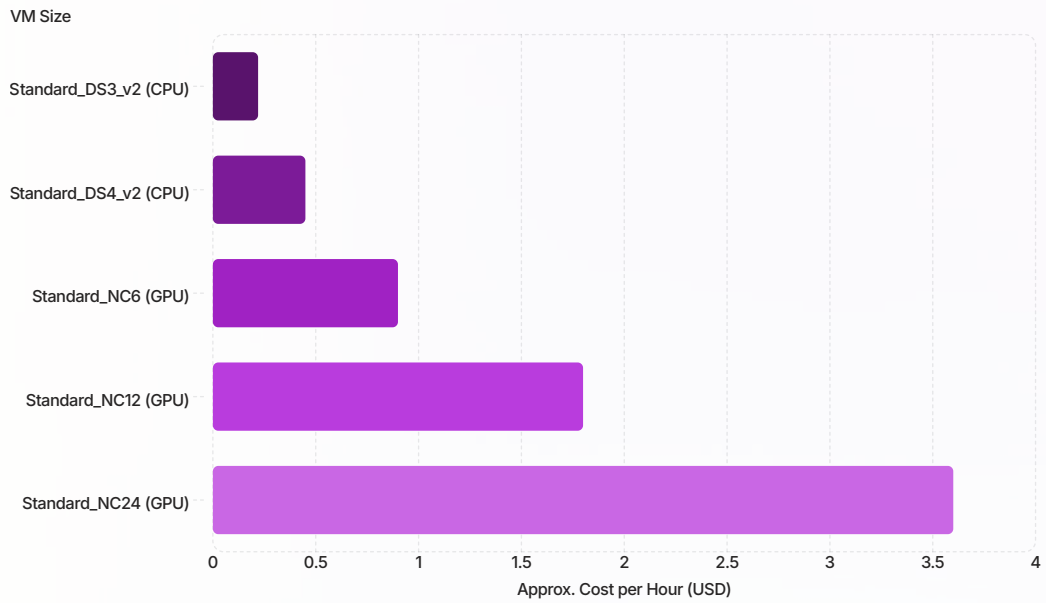
🛑 Stop Compute After Each Lab

Set `min_node_count=0` on your cluster so it scales to zero when idle. Or delete it manually — recreation takes ~2 minutes.

📅 Disable Schedules After Lab 12

Scheduled pipelines will keep firing — and billing compute — unless you explicitly disable or delete them after the session.

Compute Cost Reference

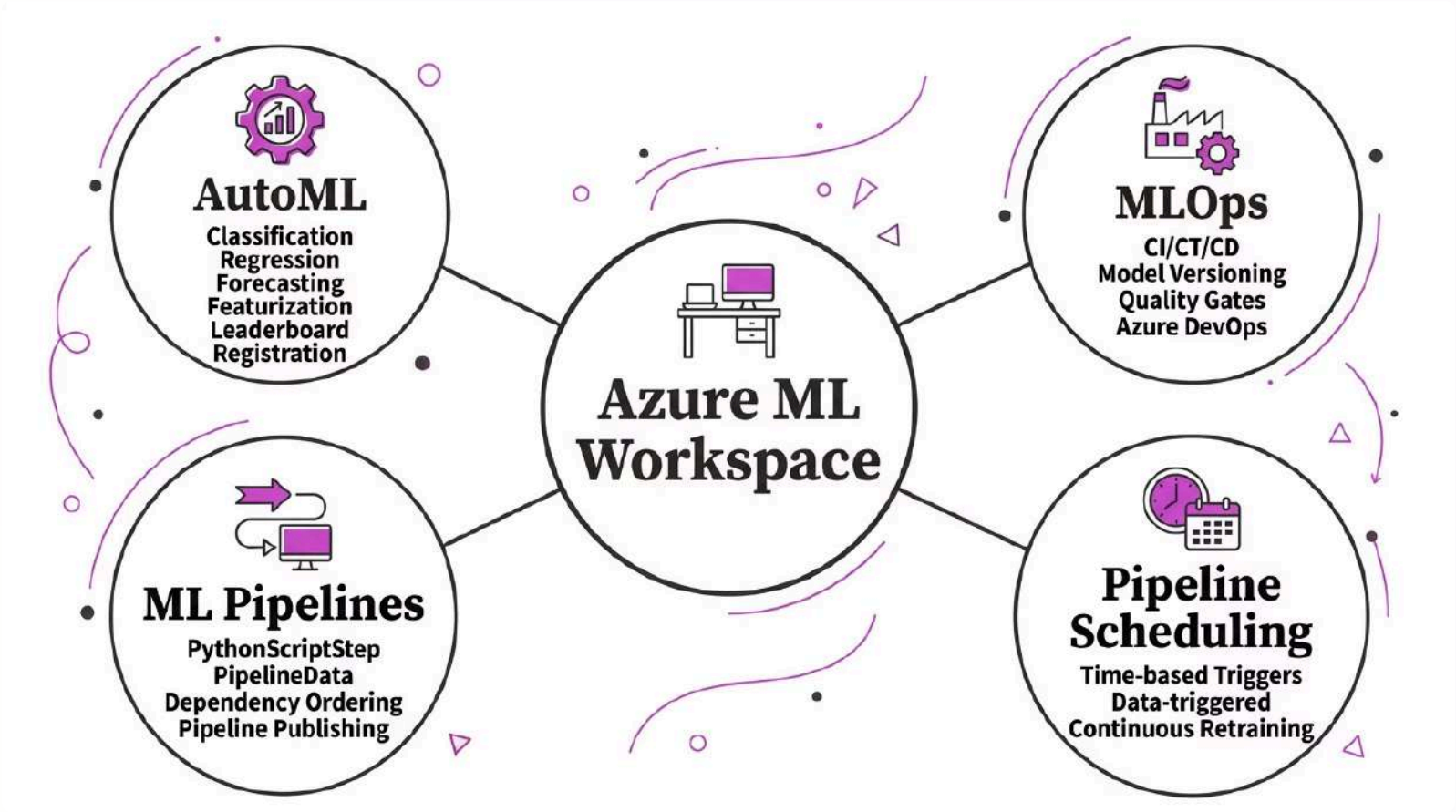


Lab Recommendation

For all Day 4 labs, use **Standard_DS3_v2** with a 2-node cluster maximum. A full 3-hour lab session costs approximately **$1.30 or less** with disciplined compute management.

⚠️ **Always verify** your cluster is at 0 nodes in Azure ML Studio before ending the session. Idle clusters with `min_node_count=1` continue billing 24/7.

Day 4 Concept Map



Every concept taught in Day 4 is a building block of a production-grade MLOps platform — all orchestrated within the Azure ML Workspace.

KEY TAKEAWAYS

Day 4 — What You've Learned

Key Takeaways: AutoML

AutoML automates the model selection and tuning process

Configure once, let Azure ML sweep dozens of algorithms and hyperparameter combinations in parallel.

Three task types cover most ML business problems

Classification, Regression, and Forecasting — each with purpose-built algorithm pools and metrics.

Always set a timeout and cap iterations for cost control

`experiment_timeout_minutes` is your most important cost guardrail in production and labs alike.

Key Takeaways: ML Pipelines

Pipelines make ML workflows reproducible and reusable

Each step is independently versioned, cached, and runnable on its own compute target.

PipelineData is the glue between steps

Passing outputs as inputs creates the dependency graph Azure ML uses to determine execution order.

Publish pipelines to enable triggering via REST or schedule

A published pipeline is a first-class REST endpoint — invoke it from any system, CI/CD tool, or schedule.

Key Takeaways: Scheduling & MLOps

Scheduled pipelines enable continuous model retraining

Daily or event-driven retraining keeps models accurate as data distributions shift over time.

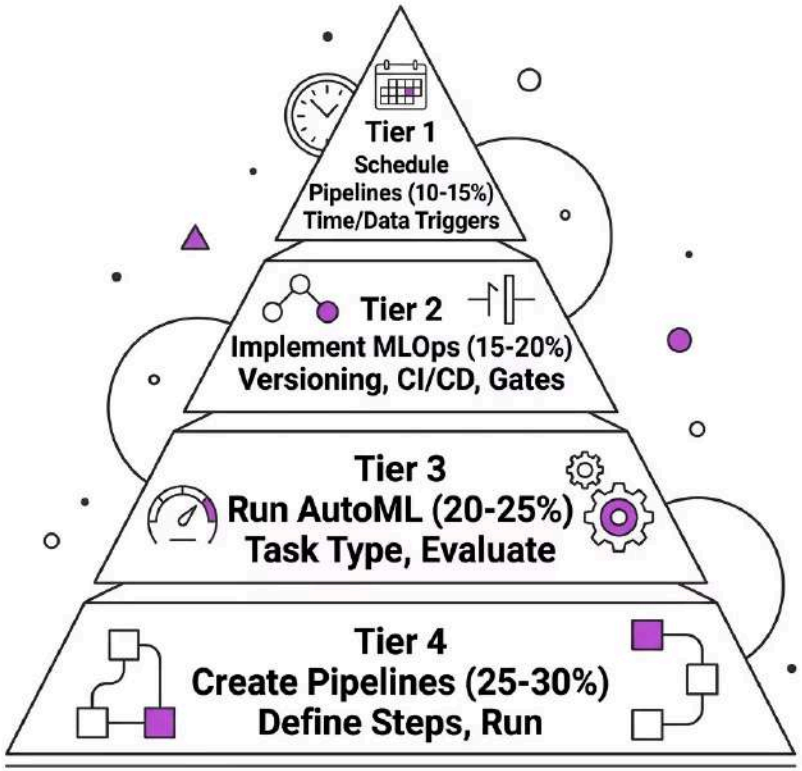
MLOps applies DevOps discipline to ML systems

CI/CT/CD for ML ensures models are tested, versioned, deployed, and monitored automatically.

Azure ML is the unified platform for all of it

Workspace, registry, pipelines, schedules, and endpoints — everything integrates natively in Azure ML.

DP-100 Exam: Day 4 Coverage



What to Focus On for the Exam

- Know the **AutoML configuration parameters** and what each controls
- Understand **PipelineData** — how outputs become inputs
- Know the difference between **time-based and data-triggered** schedules
- Understand the **model registry lifecycle** — register, stage, deploy
- Be able to identify which **MLOps practice** solves a given scenario

What's Coming: Day 5 Preview

Model Deployment

Deploy trained models to real-time and batch endpoints using ACI and AKS

Model Monitoring

Detect data drift and model degradation in production with Azure ML monitoring

DP-100 Exam Prep

Practice questions, gap analysis, and final exam strategy for DP-100 certification

📘 **Before Day 5:** Complete any unfinished Labs 10–12 and review the Azure ML Pipelines documentation at docs.microsoft.com/azure/machine-learning

Questions & Discussion

Day 4 — Automated ML · Pipelines · Scheduling · MLOps

🤔 How would you apply AutoML to a problem in your current role?

🔁 Which pipeline step would benefit most from caching in your use case?

🚀 What would it take to move your team from MLOps Level 0 to Level 2?